

Action Graph Layer —

apm_connectors/graph.py

The Python constructs this file adds on top of the FastAPI-layer doc, a from-zero LangGraph tutorial, then a complete deep dive into this repo's approval-gated action graph — written as a standalone reference, including for interview prep.

Part 0 — Python syntax this file adds

This continues the FastAPI-layer doc's Part 0 (imports, functions/defaults, type hints, classes, decorators, f-strings, comprehensions, exception chaining — read that first if any of those are still new). graph.py leans on five constructs that document didn't need to cover. Each gets a toy example, then the real line from this file.

0.1 Closures — a function that remembers variables from where it was created

This is the single most important Python idea in this file — all three of the graph's nodes are built this way. A closure is an inner function, defined inside an outer function, that keeps access to the outer function's local variables even after the outer function has returned:

```
def make_multiplier(factor):          # outer function -- called once, with a specific factor
    def multiply(x):                 # inner function -- defined fresh each time make_multiplier
        runs
        return x * factor           # `factor` isn't a global or a parameter of multiply --
    return multiply                  # it's "remembered" from the enclosing call

double = make_multiplier(2)
triple = make_multiplier(3)
double(5)    # 10
triple(5)    # 15 -- double and triple each remember a *different* factor
```

graph.py uses exactly this shape to build each node, so the node function has access to state_store (or tools) without those having to be threaded through LangGraph's own call signature, which only ever passes a node one argument — the current state:

```
# outer function -- called once, at graph-build time
def _propose_node(state_store: StateStore):
    # inner function -- this is the actual node LangGraph calls
    def propose_node(state: GraphState) -> dict[str, Any]:
        ...
        state_store.add_pending_action(...) # remembered from the closure
        ...
    return propose_node # return the closure itself, not its result

# _propose_node(state_store) runs once, right here, producing the closure
# that LangGraph will then call repeatedly, once per invocation, later
graph.add_node("propose", _propose_node(state_store))
```

Notice `_propose_node(state_store)` is *called* immediately, right there in `add_node(...)` — what gets registered as the node is its *return value*, the inner `propose_node` function, not `_propose_node` itself. This is the exact same "take dependencies as parameters, return a built thing" factory shape the FastAPI doc covered for `create_app()` — closures are just Python's version of that idea applied at function scope instead of class/app scope.

0.2 TypedDict — a dict with a declared, checkable shape

```
from typing import TypedDict

class GraphState(TypedDict, total=False):
    process_id: str
    decision: bool | None
```

A `TypedDict` looks like a class but behaves like a plain dict at runtime — `GraphState` instances are ordinary dicts (`{"process_id": "...", "decision": True}`), just with a type checker able to verify which keys are expected and what type each holds. `total=False` means no key is required — every field is optional, matching how this graph's nodes each only ever set a few keys at a time (§2.1). This is purely a static-typing aid; nothing about it changes how the dict behaves when the program actually runs.

0.3 @dataclass — a quick way to define a plain data-holding class

```
from dataclasses import dataclass

@dataclass(frozen=True)
class RunOutcome:
    process_id: str
    summary: str | None
```

Without `@dataclass`, defining a simple "bag of fields" class means hand-writing `__init__` (to accept and store each field), `__repr__` (so printing the object is readable), and `__eq__` (so two instances with equal fields compare equal). `@dataclass` generates all of that automatically from the type-hinted attributes — the same field-declaration style as a `Pydantic BaseModel` (FastAPI doc §0.4), but from Python's standard library, with no validation performed. `frozen=True` makes instances immutable — attempting `outcome.process_id = "x"` after construction raises an error, which suits `RunOutcome` since it's a finished result handed back to a caller, never meant to be mutated afterward.

0.4 getattr() — looking up a method by its name at runtime

```
tool.send_email(...)                # normal method call -- name is fixed in the source

method_name = "send_email"
method = getattr(tool, method_name)  # equivalent, but the name comes from a variable
method(...)
```

`getattr(obj, name)` fetches an attribute (here, a bound method) off an object using a string for its name, instead of writing the name directly in the source with a dot. `execute_node` uses exactly this to call whichever method a proposed action named, without needing an `if/elif` chain checking every possible method name: `method = getattr(tool, proposed["method"])` — `proposed["method"]` is a plain string like `"send_email"` that arrived from outside the process (ultimately, from an API request body).

0.5 ** in a function call — spreading a dict as keyword arguments

```
def send(to, subject, body):
    ...

payload = {"to": "a@b.com", "subject": "Hi", "body": "..."}
send(**payload)  # exactly the same as: send(to="a@b.com", subject="Hi", body="...")
```

`**some_dict` in a call "unpacks" the dict into individual keyword arguments — each key becomes a parameter name, each value the argument passed for it. This is how `execute_node` calls a method whose exact parameter list it doesn't know in advance: `method(process_id, dry_run=False, **proposed["payload"])` passes `process_id` positionally, `dry_run=False` explicitly, and then spreads whatever fields the original request body had (`to/subject/body` for a Gmail send, `object_name/fields` for a Salesforce create) as the

rest — one line handles every connector's write method, whatever its specific parameters are.

Part 1 — LangGraph from zero

1.1 What problem LangGraph solves

A plain Python function runs start to finish in one go — call it, get a result. That breaks down the moment a process needs to *pause partway through, wait for something external (here, a human decision), and resume later* — possibly in a completely different process, minutes or hours afterward, with no Python call stack still sitting around waiting. LangGraph is a library for building exactly that kind of multi-step process: a **graph** of named **nodes** (each a small function), connected by **edges** that say which node runs next, with built-in support for a node to pause the entire graph and resume it later from a durable snapshot instead of a live stack frame. This repo uses none of LangGraph's fancier features (branching, cycles, multi-agent orchestration) — just the pause/resume mechanism, applied to exactly one purpose: the human-approval gate.

1.2 Building a graph: nodes, edges, entry point

```
from langgraph.graph import END, StateGraph

graph = StateGraph(GraphState)          # GraphState: the shape of data nodes share (§1.3)

graph.add_node("propose", propose_node) # register a node under a name
graph.add_node("approval", approval_node)
graph.add_node("execute", execute_node)

graph.set_entry_point("propose")        # where a fresh run starts
graph.add_edge("propose", "approval")   # after propose, always go to approval
graph.add_edge("approval", "execute")
graph.add_edge("execute", END)          # END is a sentinel meaning "the graph is finished"

compiled = graph.compile(checkpointer=checkpointer) # turns the graph into something runnable
```

This repo's graph is deliberately the simplest possible shape: three nodes, wired in a straight line, no branching or loops. `add_edge(a, b)` is an *unconditional* edge — always go from a to b (LangGraph also supports conditional edges that pick the next node based on state, not used here). `compile()` is what actually produces a runnable object — the un-compiled graph is just a description.

1.3 State: the shared data every node reads and updates

Every node function takes the current state (a `GraphState`, §0.2) as its only argument, and returns a *partial* dict of updates — LangGraph merges whatever keys a node returns into the overall state before handing it to the next node:

```
def propose_node(state: GraphState) -> dict[str, Any]:
    ...
    return {"pending_action_id": action_record["id"]} # only this one key is being updated
```

A node never sees or returns the *whole* state unless it needs to — it reads whichever keys it cares about off the state parameter, and returns only the keys it's changing. This is why `propose_node` can return just `{"pending_action_id": ...}` without also repeating `process_id` or `proposed_action` — those stay whatever they already were.

1.4 The core mechanism: `interrupt()` and `Command(resume=...)`

This is LangGraph's actual value proposition for this codebase, and worth understanding precisely. Calling `interrupt(payload)` inside a node does two things at once: it **stops the graph's execution right there** — no later node runs — and it **surfaces** payload **back to whoever called the graph**, as a signal that it's now paused and waiting. The graph stays paused indefinitely — seconds, hours, across a server restart if the checkpointer is durable (§1.5) — until something calls the graph again with a `Command(resume=some_value)` instead of a normal input. When that happens, execution picks back up essentially at the `interrupt()` call, which now simply *returns* `some_value` instead of pausing again, and the node keeps running from there.

```
def approval_node(state: GraphState) -> dict[str, Any]:
    ...
    decision = interrupt({"type": "approval_request", "action_id": action_id, ...})
    # <-- graph pauses HERE on first run, resumes HERE (with `decision` set) on the second
    approved = bool(decision.get("approved")) if isinstance(decision, dict) else bool(decision)
    ...
```

Two separate calls into the compiled graph make this whole cycle work — `graph.invoke(initial_state, config=...)` the first time, then later `graph.invoke(Command(resume=decision), config=...)` — and both calls must pass the *same* config (specifically, the same `thread_id`, §1.5), which is how LangGraph knows which paused run to resume.

1.5 Checkpointer and `thread_id` — where paused state actually lives

A **checkpointer** is where LangGraph saves a graph's state every time it pauses (or finishes) a step, keyed by a **`thread_id`** you supply — a paused run is identified purely by that id, not by any live Python object. This repo passes `process_id` as the `thread_id`:

```
def _config(process_id: str) -> dict[str, Any]:
    return {"configurable": {"thread_id": process_id}}
```

The checkpointer implementation determines whether a pause survives a process restart. `MemorySaver` keeps checkpoints in a plain Python dict in RAM — fast, zero setup, but gone the instant the process exits. `PostgresSaver` writes them to a real database instead, so a paused approval genuinely survives a redeploy. This repo's `api/dependencies.py` picks between the two based on whether `DATABASE_URL` is set — the graph code here never changes either way, since `build_action_graph` just takes whatever checkpointer it's handed (§2.4).

1.6 Replay: the detail that shapes how every node here is written

This is the single most important, least obvious LangGraph fact, and it's the reason `approval_node` is written the way it is. **On resume, LangGraph does not continue a suspended Python stack frame** — there isn't one to continue; the process that paused may not even be the process that resumes. Instead, it **re-runs the entire interrupted node function again, from its first line**, using the checkpointed state — and the only thing different this second time is that the specific `interrupt(...)` call that previously paused now immediately returns the resume value instead of pausing again.

"On resume, LangGraph re-runs this node function from the top; `interrupt()` is the first (and only) side-effecting statement here so that replay is harmless — everything above it is a plain read of already-checkpointed state, and everything below it (recording the decision) executes exactly once, on the resume pass."

Concretely for `approval_node`: on the *first* run, execution reaches `interrupt(...)` and stops there — none of the lines after it have run yet. On *resume*, LangGraph runs the function again from line one — re-reading `proposed` and `action_id` off state (harmless, since those are just reads) — until it reaches the same `interrupt(...)` call again, which this time returns immediately with the decision instead of pausing, and *only*

then do the lines below it — `resolve_pending_action(...)` and the return — actually execute, for the first and only time. If this node did anything with a side effect *before* the `interrupt()` call, replay would silently repeat that side effect on every resume — which is exactly why `propose_node`, the one thing that must fire exactly once, is a completely separate node that never calls `interrupt()` at all, and therefore is never replayed (§2.6).

Part 2 — This repo's action graph, section by section

`graph.py` is one file, ~205 lines. Read it in this order — each section only makes sense once you've seen the one before it:

Section	What it defines
<code>GraphState</code>	The TypedDict every node reads and partially updates (§1.3).
<code>RunOutcome</code>	The caller-facing result — either paused (<code>pending_action</code> set) or done (<code>final_result</code> set).
<code>_propose_node</code> / <code>_approval_node</code> / <code>_execute_node</code>	The three closure factories building the graph's actual nodes.
<code>build_action_graph</code>	Wires the three nodes into a compiled, runnable graph.
<code>start_action</code> / <code>resume_process</code>	The only two public entry points — everything else is private (leading underscore).
<code>_to_outcome</code>	Converts <code>LangGraph</code> 's raw <code>invoke()</code> result into this repo's <code>RunOutcome</code> shape.

2.1 GraphState — six optional keys, total=False

```
class GraphState(TypedDict, total=False):
    process_id: str
    category: str
    proposed_action: dict[str, Any] | None
    pending_action_id: str | None
    decision: bool | None
    result: dict[str, Any] | None
```

Each field maps to exactly one moment in the pipeline: `proposed_action` is set once, at the very start, by the caller (§2.5); `pending_action_id` is set by `propose_node`; `decision` by `approval_node`; `result` by `execute_node`. No node ever needs to see the whole shape at once — each just reads the keys the node before it promised to set.

2.2 RunOutcome — exactly one of two fields is ever set

```
@dataclass(frozen=True)
class RunOutcome:
    """Either the graph is paused waiting for a human decision
    (pending_action set), or it has finished (final_result set)."""
    process_id: str
    summary: str | None
    pending_action: dict[str, Any] | None
    final_result: dict[str, Any] | None
```

This is the exact same "discriminated by convention" shape the FastAPI doc's `RunOutcomeResponse` mirrors one-to-one (that Pydantic model is literally built by copying these same four fields off a `RunOutcome` in `_responses.to_response`) — a caller checks which field is non-None rather than the code raising a different

exception or return type per case.

2.3 The three node factories

`propose_node` — runs to completion, never interrupts, never replayed:

```
def _propose_node(state_store: StateStore):
    def propose_node(state: GraphState) -> dict[str, Any]:
        process_id = state["process_id"]
        proposed = state.get("proposed_action")
        if not proposed:
            return {"pending_action_id": None}
        action_record = state_store.add_pending_action(
            process_id=process_id, tool=proposed["tool"], description=proposed["description"],
            payload=proposed, category=state.get("category", "other"),
        )
        return {"pending_action_id": action_record["id"]}
    return propose_node
```

`approval_node` — the gate itself (walked through fully in §1.6):

```
def _approval_node(state_store: StateStore):
    def approval_node(state: GraphState) -> dict[str, Any]:
        proposed = state.get("proposed_action")
        action_id = state.get("pending_action_id")
        if not proposed or not action_id:
            return {"decision": None}
        decision = interrupt({"type": "approval_request", "action_id": action_id,
                              "tool": proposed["tool"], "method": proposed["method"],
                              "description": proposed["description"], "payload": proposed["payload"],
                              "category": state.get("category", "other")})
        approved = bool(decision.get("approved")) if isinstance(decision, dict) else
            bool(decision)
        state_store.resolve_pending_action(action_id, approved=approved)
        return {"decision": approved}
    return approval_node
```

`execute_node` — the only place a write ever actually fires:

```
def _execute_node(tools: dict[str, BaseTool], state_store: StateStore):
    def execute_node(state: GraphState) -> dict[str, Any]:
        process_id = state["process_id"]
        proposed = state.get("proposed_action")
        decision = state.get("decision")
        if not proposed or not decision:
            outcome = {"executed": False, "reason": "no_action_proposed" if not proposed else
                       "rejected"}
            state_store.set_status(process_id, stage="done", result=outcome)
            return {"result": outcome}
        tool = tools[proposed["tool"]]
        method = getattr(tool, proposed["method"])
        action_result = method(process_id, dry_run=False, **proposed["payload"])
        outcome = {"executed": action_result.executed, "description": action_result.description,
                   "details": action_result.details}
        state_store.set_status(process_id, stage="done", result=outcome)
        return {"result": outcome}
    return execute_node
```

`dry_run=False` is hardcoded, always, unconditionally, right here — there is no code path anywhere in this file that can call a tool's write method with `dry_run=True` or omit the flag. Combined with the fact that this is the *only* place in the entire codebase any write/action method is ever invoked, that one hardcoded keyword argument is effectively the codebase's whole safety guarantee, expressed as a single line.

2.4 build_action_graph — wiring it together

```
def build_action_graph(tools: dict[str, BaseTool], state_store: StateStore, checkpointer: Any):
    graph = StateGraph(GraphState)
    graph.add_node("propose", _propose_node(state_store))
    graph.add_node("approval", _approval_node(state_store))
    graph.add_node("execute", _execute_node(tools, state_store))
    graph.set_entry_point("propose")
    graph.add_edge("propose", "approval")
    graph.add_edge("approval", "execute")
    graph.add_edge("execute", END)
    return graph.compile(checkpointer=checkpointer)
```

checkpointer is a required parameter here, not defaulted to `MemorySaver()` inside this function — the module docstring is explicit that this is deliberate, "so callers decide the persistence story." This is the same Strategy-pattern shape the FastAPI doc's `dependencies.py` section covered: the decision of *which* checkpointer lives entirely at the call site (`api/dependencies.py`'s `get_action_graph()`), never inside `graph.py` itself.

2.5 start_action / resume_process — the only two public entry points

```
def start_action(graph, process_id, tool, method, description, payload, category="manual") -> RunOutcome:
    proposed_action = {"tool": tool, "method": method, "description": description, "payload": payload}
    initial_state: GraphState = {"process_id": process_id, "proposed_action": proposed_action, "category": category}
    result = graph.invoke(initial_state, config=_config(process_id))
    return _to_outcome(process_id, result)

def resume_process(graph, process_id, approved: bool) -> RunOutcome:
    result = graph.invoke(Command(resume={"approved": approved}), config=_config(process_id))
    return _to_outcome(process_id, result)
```

Every other function in this file has a leading underscore (`_propose_node`, `_config`, `_to_outcome`) — Python's convention for "internal, not part of this module's public interface," enforced by nothing but the reader's own discipline (there's no access-control keyword like `private` in Python). `start_action` and `resume_process` are the two callers actually use — `tools_routes.py`'s `_propose` helper calls the former, its `decide_action` route calls the latter (see the FastAPI-layer doc §2.4).

2.6 _to_outcome — reading LangGraph's own sentinel key

```
def _to_outcome(process_id: str, result: dict[str, Any]) -> RunOutcome:
    if "__interrupt__" in result:
        return RunOutcome(process_id=process_id, summary=result.get("summary"),
                           pending_action=result["__interrupt__"][0].value, final_result=None)
    return RunOutcome(process_id=process_id, summary=result.get("summary"),
                       pending_action=None, final_result=result.get("result"))
```

`graph.invoke(...)` returns a plain dict either way — LangGraph communicates "this run just paused" by including a special `"__interrupt__"` key in that dict (a list of interrupt objects; `[0].value` is the payload the paused `interrupt(...)` call was given). This function is the one place in the codebase that reads that LangGraph-specific detail and translates it into this repo's own vocabulary (`RunOutcome.pending_action`) — everything above `graph.py`, all the way out through the API layer, only ever deals with `RunOutcome`, never LangGraph's own result shape directly.

Part 3 — Design patterns used here, named explicitly

Pattern	Where	What problem it actually solves here
Closure-based Factory	<code>_propose_node/_approval_node/_execute_node</code>	Gives each node access to <code>state_store/tools</code> without LangGraph's call signature needing to pass them — the closure remembers them from build time (§0.1).
State Machine / Pipeline	The 3-node graph: <code>propose -> approval -> execute</code>	Models a multi-step process with an explicit, inspectable shape instead of one long function — each stage has a name and a single responsibility.
Command Pattern	<code>Command(resume={"approved": approved})</code>	Wraps "resume this paused run with this value" as a single object passed to <code>invoke()</code> , instead of a special-cased <code>resume()</code> method with its own signature.
Memento / Checkpoint	The checkpointer (<code>MemorySaver / PostgresSaver</code>)	Captures a paused run's full state externally, so it can be restored later by an unrelated process invocation, not just resumed within the same call stack.
Strategy (pluggable implementation)	<code>build_action_graph's</code> checkpointer parameter	The graph never picks its own persistence — the caller decides in-memory vs. durable, matching the FastAPI doc's <code>DATABASE_URL</code> toggle.
Idempotency by construction	<code>propose_node</code> never calls <code>interrupt()</code> ; <code>approval_node's</code> side effect sits after it	Guarantees a pending action is recorded exactly once and a decision resolved exactly once, despite LangGraph replaying interrupted nodes on every resume (§1.6).
Facade	<code>start_action / resume_process</code>	Two public functions hide the graph-invocation details (<code>Command</code> objects, <code>thread_id</code> config, the <code>__interrupt__</code> sentinel) from every caller.

Part 4 — Interview-prep Q&A

Practice answering by pointing at the actual code, not reciting the definition.

Q: What does `interrupt()` actually do, mechanically?

A: It stops the graph's execution at that point and surfaces its payload to the caller as a signal that the run is paused. The graph doesn't keep a live, waiting Python stack frame -- the paused state is written to the checkpointer, and a completely separate later call (`graph.invoke(Command(resume=...))`) is what continues it (§1.4).

Q: What happens to the code before `interrupt()` when a node resumes -- does it run again?

A: Yes -- LangGraph replays the entire node function from its first line on resume; only the specific `interrupt()` call that previously paused now returns immediately instead of pausing again. Every node in this file is written assuming that replay, which is why `interrupt()` is deliberately the first side-effecting statement in `approval_node` (§1.6).

Q: Why is there a separate `propose_node` instead of recording the pending action inside `approval_node`, right before its `interrupt()` call?

A: Because `approval_node` gets replayed on resume and `propose_node` doesn't. If recording the pending action happened just before `interrupt()` inside the same node, it would re-run (and double-record) every time that node replays. Splitting it into its own node that never calls `interrupt()` guarantees it fires exactly once (§2.6, the module's own docstring makes this explicit).

Q: What is a checkpointer, and what's the practical difference between MemorySaver and PostgresSaver?

A: It's where LangGraph persists a paused (or finished) run's state, keyed by `thread_id`. MemorySaver keeps it in an in-process dict -- fast, zero setup, gone on restart. PostgresSaver writes it to a real database, so a paused approval survives a redeploy or a crash -- this repo's live-verified scenario for exactly that (§1.5).

Q: What is `thread_id` here, and where does it come from?

A: It's the key LangGraph uses to know which paused run a given `invoke()` call is resuming. This repo passes `process_id` as the `thread_id` (`_config()`) -- the same id used for the API's `action_id` and for audit-trail grouping, so one id ties the whole flow together end to end.

Q: `getattr(tool, proposed['method'])` -- why not just call the method directly, e.g. `tool.send_email(...)`?

A: `execute_node` is generic across every connector and every write method -- it doesn't know at write-time whether it's handling a Gmail send, a Salesforce create, or a Jira update. `getattr` looks the method up by the name that arrived in the request instead of the code needing an `if/elif` branch per method (§0.4).

Q: Where, precisely, is `dry_run=False` set, and why does that matter?

A: One hardcoded keyword argument inside `execute_node`'s call: `method(process_id, dry_run=False, **proposed['payload'])`. Combined with `execute_node` being the only place in the entire codebase any write/action method is ever called, that single line is effectively the whole enforcement mechanism, backed up independently by each connector's own `require_dry_run_guard` (see the FastAPI-layer doc).

Q: Walk through what happens end to end when a write is proposed and later approved.

A: `start_action` builds the initial `GraphState` and calls `graph.invoke` -- `propose_node` records the pending action and returns `pending_action_id`; `approval_node` calls `interrupt()`, pausing the graph; `_to_outcome` sees `__interrupt__` in the result and returns a `RunOutcome` with `pending_action` set. Later, `resume_process` calls `graph.invoke(Command(resume={'approved': True}))` with the same `thread_id`; `approval_node` replays, `interrupt()` now returns immediately, `resolve_pending_action` logs the approval, `execute_node` runs the real write, and `_to_outcome` returns a `RunOutcome` with `final_result` set instead.

Vocabulary cheat-sheet

Term	Meaning here
Closure	An inner function that keeps access to variables from the outer function it was defined in, even after the outer call returns (§0.1).
TypedDict	A dict with a statically-declared, checkable key/value shape -- behaves as a plain dict at runtime (§0.2).
@dataclass	Standard-library decorator that auto-generates <code>__init__</code> / <code>__repr__</code> / <code>__eq__</code> from type-hinted attributes (§0.3).
getattr(obj, name)	Looks up an attribute (e.g. a method) on an object using a string name instead of a fixed dot-name (§0.4).
**dict unpacking	Spreads a dict's items as individual keyword arguments in a function call (§0.5).
Node / Edge	A node is one step (a function); an edge says which node runs next after it.
StateGraph / compile()	<code>StateGraph</code> describes the graph; <code>compile()</code> turns that description into a runnable object.

interrupt()	Pauses graph execution at that exact point and surfaces its argument to the caller (§1.4).
Command(resume=...)	What you pass to invoke() to resume a paused run with a value for the interrupt() call to return.
Checkpoint / thread_id	Where a paused run's state is stored, keyed by an id you supply -- MemorySaver (in-process) or PostgresSaver (durable) here.
Replay	LangGraph re-running an interrupted node from its top on resume -- the reason node code must be written to tolerate repeated execution before its interrupt() call (§1.6).

That's the whole layer: one file, three nodes, a straight-line graph, and one interrupt() call that is the entire reason this package can promise a write never executes without a separate, later, human-approved decision -- durable across process restarts whenever a real checkpoint backs it.